COMPLETE MASTER GUIDE

JavaScript

From Zero to Confident Developer

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

WHAT THIS COMPLETE GUIDE COVERS

Table of Contents

Table of Contents

Chapter 1 — What Is JavaScript?

JavaScript is the programming language of the web. While HTML builds the structure of a webpage and CSS makes it look beautiful, JavaScript makes it come alive — responding to clicks, validating forms, loading new content, animating elements, and communicating with servers in real time.

Every major website you use today — Google, YouTube, Facebook, Safaricom, KCB — relies heavily on JavaScript. It is the only programming language that runs natively in every web browser, making it the most important language a frontend developer can learn.

1.1 A Brief History

Year	Milestone
1995	Brendan Eich creates JavaScript in just 10 days at Netscape. Originally called Mocha, then LiveScript.
1997	JavaScript is standardised as ECMAScript (ES1) by ECMA International — ensuring all browsers use the same rules.
2009	Node.js is released — JavaScript can now run on servers, not just in browsers.
2015	ES6 (ES2015) — the biggest upgrade ever: arrow functions, classes, let/const, promises, modules.
Today	JavaScript is the world's most-used language. Runs browsers, servers, mobile apps, AI, IoT devices.

1.2 What Can JavaScript Do?

Capability	What It Means	Real Example
DOM Manipulation	Change text, styles, images, and structure of a page after it loads	Show/hide a menu, update a counter
Event Handling	Respond to user actions: clicks, typing, scrolling, hovering	Button click shows a popup
Form Validation	Check inputs before submitting to the server	Alert if email is empty
Animations	Move elements, create visual effects and transitions	Fade-in hero section
AJAX / Fetch	Load data from a server without reloading the whole page	Load new tweets dynamically
Local Storage	Save data in the browser (no server needed)	Remember dark mode preference
Browser APIs	Access camera, GPS, notifications, audio, canvas	Map, photo upload, PWA

Capability	What It Means	Real Example
Server-Side (Node.js)	Run JavaScript on a server to handle requests and databases	Build a REST API

1.3 How JavaScript Runs in the Browser

1. Browser loads the HTML file and starts building the DOM (page structure).
2. When it reaches a <script> tag, it pauses HTML parsing and runs the JavaScript.
3. JavaScript is executed by the browser's JS engine (V8 in Chrome, SpiderMonkey in Firefox).
4. The JS engine compiles the code to machine instructions and runs them instantly.
5. Results appear in the browser: the page changes, an alert fires, data loads.

1.4 Adding JavaScript to a Webpage

```
<!-- METHOD 1: External file (BEST PRACTICE) -->
<!-- Place before </body> so HTML loads first -->
<script src="app.js"></script>

<!-- METHOD 2: Internal script block -->
<script>
  console.log('Hello from internal script!');
</script>

<!-- METHOD 3: Inline event (avoid — bad practice) -->
<button onclick="alert('Clicked!')">Click Me</button>

<!-- defer: loads JS file in background, runs after HTML is parsed -->
<script src="app.js" defer></script>
```

ALWAYS USE console.log() TO TEST YOUR CODE

console.log() prints a value to the browser's developer console.
Open the console: Press F12 (or Ctrl+Shift+I) → click the Console tab.
This is your most powerful debugging tool as a JavaScript developer.
Example: console.log('My name is Lucky'); → prints: My name is Lucky

Chapter 2 — Variables, Data Types & Operators

A variable is a named container that stores a value. Think of it as a labelled box — you put data inside, give the box a name, and use that name later to get the data back.

2.1 Declaring Variables: var, let, const

```
// let — block-scoped, can be reassigned (use this most of the time)
let name = 'Lucky Nakola';
name = 'James';          // ☑ Reassignment is fine

// const — block-scoped, CANNOT be reassigned (use for fixed values)
const PI = 3.14159;
// PI = 3;               // ✕ TypeError: Assignment to constant variable

// var — function-scoped, older style (AVOID in modern JS)
var city = 'Nyeri';      // Works but has confusing hoisting behaviour
```

Keyword	When to Use & Behaviour
let	Block-scoped. Can be updated. Cannot be re-declared in same scope. Use for values that change.
const	Block-scoped. Cannot be updated or re-declared. Use for values that should never change.
var	Function-scoped. Hoisted to top of function. Can be re-declared. Avoid in modern code.

2.2 Data Types

JavaScript has 8 data types. The first 7 are primitive (simple values), and the last is an Object (complex).

Type	Description	Example
String	Text, wrapped in quotes	'Lucky' "Nairobi" `Hello`
Number	Integers and decimals	42 3.14 -7 0
Boolean	True or false only	true false
null	Intentionally empty value	let score = null;
undefined	Variable declared, no value yet	let x; → x is undefined
BigInt	Very large integers	9007199254740991n
Symbol	Unique identifier (advanced)	Symbol('id')
Object	Collections of key-value pairs	{ name: 'Lucky', age: 30 }

```
// Checking the type of a value
typeof 'Lucky'       // → 'string'
typeof 42            // → 'number'
typeof true          // → 'boolean'
typeof undefined     // → 'undefined'
typeof null          // → 'object'   ← famous JS quirk (historical bug)
typeof {}            // → 'object'
typeof []            // → 'object'   ← arrays are objects in JS
typeof function(){}  // → 'function'
```

2.3 Template Literals (String Interpolation)

Template literals use backticks (`) and allow you to embed variables directly inside strings with ${}.

```
const firstName = 'Lucky';
const course     = 'JavaScript';
const year       = 2025;

// Old way (string concatenation)
console.log('Welcome, ' + firstName + '! You are studying ' + course + ' in ' +
year);

// Modern way (template literal) — much cleaner!
console.log(`Welcome, ${firstName}! You are studying ${course} in ${year}.`);
// Output: Welcome, Lucky! You are studying JavaScript in 2025.

// Multi-line strings with template literals
const card = `
  Name:  ${firstName}
  Year:  ${year}
  Score: ${90 + 5}
`;
```

2.4 Operators

Arithmetic Operators

```
let a = 20, b = 6;
console.log(a + b);   // 26  — Addition
console.log(a - b);   // 14  — Subtraction
console.log(a * b);   // 120 — Multiplication
console.log(a / b);   // 3.33 — Division
console.log(a % b);   // 2   — Modulus (remainder)
console.log(a ** 2);  // 400 — Exponentiation (a to the power of 2)
let x = 5;
x++;   // x becomes 6 — Increment
x--;   // x becomes 5 — Decrement
```

Comparison Operators

```
// == vs === — this is CRITICAL to understand
5 == '5'    // true  — loose equality (checks value only, ignores type)
5 === '5'   // false — strict equality (checks value AND type) ← ALWAYS USE THIS
5 !== '5'   // true  — strict inequality

10 > 5     // true
10 < 5     // false
10 >= 10   // true
10 <= 9    // false
```

Logical Operators

```
// AND (&&) — both conditions must be true
true && true   // true
true && false  // false

// OR (||) — at least one condition must be true
false || true  // true
false || false // false

// NOT (!) — reverses the boolean
!true   // false
!false  // true

// Practical example
const age  = 20;
const hasID = true;
if (age >= 18 && hasID) {
  console.log('Access granted.');
}
```

Assignment Operators

```
let score = 100;
score += 10;   // score = score + 10  → 110
score -= 5;    // score = score - 5   → 105
score *= 2;    // score = score * 2   → 210
score /= 3;    // score = score / 3   → 70
score %= 4;    // score = score % 4   → 2

// Nullish coalescing assignment
let username = null;
username ??= 'Guest';  // If null or undefined, assign 'Guest'
console.log(username); // 'Guest'
```

ALWAYS USE === NOT ==

Using == allows JavaScript to coerce (convert) types before comparing.
This causes hard-to-find bugs: 0 == false is TRUE, '' == false is TRUE.
Using === is strict — it never converts types — always predictable.
Rule: Use === and !== everywhere. Only use == if you have a very specific reason.

Chapter 3 — Control Flow: Conditionals & Loops

Control flow is the order in which JavaScript executes code. Conditionals let you make decisions. Loops let you repeat code. Together they are the logic engine of every program.

3.1 if / else if / else

```
const score = 75;

if (score >= 90) {
  console.log('Grade: A — Excellent!');
} else if (score >= 75) {
  console.log('Grade: B — Good work!');
} else if (score >= 60) {
  console.log('Grade: C — Keep improving.');
} else {
  console.log('Grade: F — Please resit.');
}
// Output: Grade: B — Good work!
```

3.2 The Ternary Operator — Shorthand if/else

```
// Syntax: condition ? valueIfTrue : valueIfFalse
const age = 20;
const access = age >= 18 ? 'Granted' : 'Denied';
console.log(access);  // 'Granted'

// Practical use — inline in a template literal
const user = { name: 'Lucky', isPremium: true };
console.log(`Hello ${user.name}, you are a ${user.isPremium ? 'Premium' : 'Free'}
member.`);
// Output: Hello Lucky, you are a Premium member.
```

3.3 switch Statement

```
const day = 'Monday';

switch (day) {
  case 'Monday':
  case 'Tuesday':
  case 'Wednesday':
  case 'Thursday':
  case 'Friday':
    console.log('Weekday — class is on!');
```

```
    break;
  case 'Saturday':
    console.log('Saturday — workshop day!');
    break;
  case 'Sunday':
    console.log('Sunday — rest day.');
    break;
  default:
    console.log('Unknown day.');
}
```

3.4 Loops

for Loop — when you know how many times to repeat

```
for (let i = 1; i <= 5; i++) {
  console.log(`Lap ${i} complete.`);
}
// Lap 1 complete.
// Lap 2 complete. ... Lap 5 complete.

// Loop backwards
for (let i = 10; i >= 1; i--) {
  console.log(i);
}
```

while Loop — repeat while a condition is true

```
let attempts = 0;

while (attempts < 3) {
  console.log(`Login attempt ${attempts + 1}`);
  attempts++;
}
console.log('Too many attempts. Account locked.');
```

do...while Loop — always runs at least once

```
let password;

do {
  password = prompt('Enter your password:');
} while (password !== 'secret123');

console.log('Password accepted!');
```

for...of — iterate over array values

```
const students = ['Alice', 'Bob', 'Carol', 'David'];
```

```
for (const student of students) {
  console.log(`Welcome, ${student}!`);
}
```

for...in — iterate over object keys

```
const profile = { name: 'Lucky', city: 'Nyeri', role: 'Instructor' };

for (const key in profile) {
  console.log(`${key}: ${profile[key]}`);
}
// name: Lucky
// city: Nyeri
// role: Instructor
```

break and continue

```
// break — exit the loop immediately
for (let i = 0; i < 10; i++) {
  if (i === 5) break;
  console.log(i);    // 0 1 2 3 4
}

// continue — skip this iteration and move to the next
for (let i = 0; i < 6; i++) {
  if (i === 3) continue;
  console.log(i);    // 0 1 2 4 5  (3 is skipped)
}
```

Chapter 4 — Functions

A function is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you write it once in a function and call it whenever you need it. Functions are the foundation of organized, efficient JavaScript.

4.1 Function Declaration

```
// Declaring a function
function greet(name) {
  return `Hello, ${name}! Welcome to JavaScript.`;
}

// Calling the function
console.log(greet('Lucky'));   // Hello, Lucky! Welcome to JavaScript.
console.log(greet('Alice'));   // Hello, Alice! Welcome to JavaScript.

// Function with multiple parameters
function add(a, b) {
  return a + b;
}
console.log(add(10, 25));   // 35
```

4.2 Function Expression

```
// Assigning a function to a variable
const multiply = function(a, b) {
  return a * b;
};

console.log(multiply(4, 6));   // 24

// Note: Function declarations are HOISTED (can be called before they appear)
// Function expressions are NOT hoisted
```

4.3 Arrow Functions (ES6)

Arrow functions are a shorter, cleaner way to write functions. They are one of the most-used features of modern JavaScript.

```
// Traditional function
function square(n) {return n * n;}

// Arrow function — same thing, shorter syntax
const square = (n) => {return n * n; };
```

```
// Even shorter: if one parameter, remove parentheses
const square = n => { return n * n; };

// Even shorter: if body is just a return, remove braces and 'return'
const square = n => n * n;

console.log(square(5));   // 25

// Arrow function with multiple parameters
const fullName = (first, last) => `${first} ${last}`;
console.log(fullName('Lucky', 'Nakola'));   // Lucky Nakola

// Arrow function with no parameters
const getYear = () => new Date().getFullYear();
console.log(getYear());   // 2025
```

4.4 Default Parameters

```
function greet(name = 'Guest', greeting = 'Hello') {
  return `${greeting}, ${name}!`;
}

console.log(greet('Lucky', 'Welcome'));   // Welcome, Lucky!
console.log(greet('Alice'));              // Hello, Alice!
console.log(greet());                    // Hello, Guest!
```

4.5 Rest Parameters & Spread Operator

```
// Rest — collect multiple arguments into an array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4, 5));   // 15

// Spread — expand an array into individual values
const nums = [3, 7, 1, 9, 4];
console.log(Math.max(...nums));     // 9

const a = [1, 2, 3];
const b = [4, 5, 6];
const combined = [...a, ...b];
console.log(combined);  // [1, 2, 3, 4, 5, 6]
```

4.6 Scope: Where Variables Live

```
// Global scope — accessible everywhere
const globalMessage = 'I am global';

function showScope() {
  // Function scope — only accessible inside this function
  const funcMessage = 'I am in the function';

  if (true) {
    // Block scope — only accessible inside this {} block
    let blockMessage = 'I am in the block';
    console.log(globalMessage);   // ☑ Works
    console.log(funcMessage);     // ☑ Works
    console.log(blockMessage);    // ☑ Works
  }
  // console.log(blockMessage);  // ✗ ReferenceError — out of scope
}

showScope();
// console.log(funcMessage);     // ✗ ReferenceError — out of scope
```

4.7 Higher-Order Functions & Callbacks

```
// A higher-order function takes another function as an argument
function runTask(taskName, callback) {
  console.log(`Starting: ${taskName}`);
  callback();
  console.log(`Finished: ${taskName}`);
}

runTask('HTML Quiz', () => {
  console.log('Answering quiz questions...');
});
// Starting: HTML Quiz
// Answering quiz questions...
// Finished: HTML Quiz
```

Chapter 5 — Arrays

An array is an ordered list of values stored in a single variable. Arrays can hold any type of data — strings, numbers, booleans, objects, or even other arrays. They are one of the most important data structures in JavaScript.

5.1 Creating & Accessing Arrays

```
// Creating an array
const fruits = ['Mango', 'Avocado', 'Banana', 'Pineapple'];

// Accessing elements (index starts at 0)
console.log(fruits[0]);   // 'Mango'
console.log(fruits[2]);   // 'Banana'
console.log(fruits[fruits.length - 1]);   // 'Pineapple' (last item)

// Modifying an element
fruits[1] = 'Pawpaw';
console.log(fruits);   // ['Mango', 'Pawpaw', 'Banana', 'Pineapple']

// Array length
console.log(fruits.length);   // 4
```

5.2 Essential Array Methods

```
const students = ['Alice', 'Bob', 'Carol'];

// Add / Remove
students.push('David');      // Add to END → ['Alice','Bob','Carol','David']
students.pop();              // Remove from END → ['Alice','Bob','Carol']
students.unshift('Zara');    // Add to BEGINNING → ['Zara','Alice','Bob','Carol']
students.shift();            // Remove from BEGINNING → ['Alice','Bob','Carol']

// Find index and check
console.log(students.indexOf('Bob'));      // 1
console.log(students.includes('Carol'));   // true

// Slice — extract portion (does NOT modify original)
console.log(students.slice(0, 2));   // ['Alice', 'Bob']

// Splice — remove/insert elements (MODIFIES original)
students.splice(1, 1, 'James', 'Janet');
// Remove 1 element at index 1, insert 'James' and 'Janet'

// Join — array to string
console.log(students.join(' | '));   // 'Alice | James | Janet | Carol'
```

```
// Reverse and Sort
const nums = [3, 1, 4, 1, 5, 9];
nums.sort((a, b) => a - b);     // [1, 1, 3, 4, 5, 9]
nums.reverse();                 // [9, 5, 4, 3, 1, 1]
```

5.3 Array Iteration Methods (Very Important)

```
const marks = [45, 78, 92, 55, 88, 61];

// forEach — loop through each element (no return value)
marks.forEach((mark, index) => {
  console.log(`Student ${index + 1}: ${mark} marks`);
});

// map — transform each element, returns NEW array
const grades = marks.map(mark => {
  if (mark >= 80) return 'A';
  if (mark >= 60) return 'B';
  if (mark >= 50) return 'C';
  return 'F';
});
console.log(grades);   // ['F', 'B', 'A', 'C', 'A', 'B']

// filter — keep only elements that pass a test, returns NEW array
const passed = marks.filter(mark => mark >= 50);
console.log(passed);   // [78, 92, 55, 88, 61]

// find — returns the FIRST element that passes the test
const firstHigh = marks.find(mark => mark >= 90);
console.log(firstHigh);   // 92

// reduce — accumulate all elements to a single value
const total = marks.reduce((sum, mark) => sum + mark, 0);
console.log(total);   // 419
console.log(total / marks.length);   // 69.83 (average)

// every — true if ALL elements pass
console.log(marks.every(m => m >= 40));   // true

// some — true if ANY element passes
console.log(marks.some(m => m >= 90));    // true
```

Chapter 6 — Objects

An object is a collection of key-value pairs. Where an array stores an ordered list by index (0, 1, 2…), an object stores data by named keys (name, age, city…). Objects are the most powerful data structure in JavaScript — almost everything in JS is an object at a deep level.

6.1 Creating & Accessing Objects

```
// Object literal syntax
const student = {
  name: 'Lucky Nakola',
  age: 30,
  city: 'Nyeri',
  courses: ['HTML', 'CSS', 'JavaScript'],
  isEnrolled: true,
};

// Accessing properties — dot notation (preferred)
console.log(student.name);      // 'Lucky Nakola'
console.log(student.courses);   // ['HTML', 'CSS', 'JavaScript']

// Accessing properties — bracket notation (use when key is dynamic)
const key = 'city';
console.log(student[key]);   // 'Nyeri'

// Adding a new property
student.email = 'lucky@example.com';

// Updating a property
student.age = 31;

// Deleting a property
delete student.isEnrolled;
```

6.2 Object Methods

```
const calculator = {
  brand: 'MathPro',
  add: function(a, b) { return a + b; },
  subtract(a, b) { return a - b; },     // Shorthand method syntax
  multiply: (a, b) => a * b,            // Arrow function method

  // 'this' refers to the object — works with function keyword
  describe() {
    return `I am the ${this.brand} calculator.`;
  }
```

```
};

console.log(calculator.add(10, 5));     // 15
console.log(calculator.describe());    // I am the MathPro calculator.
```

6.3 Object Destructuring

```
const person = { name: 'Alice', age: 25, city: 'Mombasa', role: 'Designer' };

// Extract properties into variables
const { name, city } = person;
console.log(name);   // 'Alice'
console.log(city);   // 'Mombasa'

// Rename while destructuring
const { name: fullName, age: years } = person;
console.log(fullName);   // 'Alice'
console.log(years);      // 25

// Default value if property doesn't exist
const { country = 'Kenya' } = person;
console.log(country);   // 'Kenya'

// Destructuring in function parameters
function greetUser({ name, city }) {
  return `Hi ${name}, welcome from ${city}!`;
}
console.log(greetUser(person));   // Hi Alice, welcome from Mombasa!
```

6.4 Useful Object Methods

```
const car = { brand: 'Toyota', model: 'Prado', year: 2023 };

// Get all keys
console.log(Object.keys(car));    // ['brand', 'model', 'year']

// Get all values
console.log(Object.values(car));  // ['Toyota', 'Prado', 2023]

// Get all key-value pairs
console.log(Object.entries(car));
// [['brand','Toyota'], ['model','Prado'], ['year',2023]]

// Merge two objects (spread)
const extras = { colour: 'White', sunroof: true };
const fullCar = { ...car, ...extras };
console.log(fullCar);
// {brand:'Toyota', model:'Prado', year:2023, colour:'White', sunroof:true}
```

```
// Check if a property exists
console.log('brand' in car);   // true
console.log('price' in car);   // false
```

6.5 Arrays of Objects — The Most Common Pattern

```
const team = [
  { id: 1, name: 'Lucky',  role: 'Instructor', score: 98 },
  { id: 2, name: 'Alice',  role: 'Student',    score: 82 },
  { id: 3, name: 'Bob',    role: 'Student',    score: 74 },
  { id: 4, name: 'Carol',  role: 'Student',    score: 91 },
];

// Get all names
const names = team.map(person => person.name);
console.log(names);   // ['Lucky', 'Alice', 'Bob', 'Carol']

// Find students only (filter by role)
const students = team.filter(p => p.role === 'Student');

// Find person by ID
const found = team.find(p => p.id === 3);
console.log(found.name);   // 'Bob'

// Sort by score (highest first)
const ranked = [...team].sort((a, b) => b.score - a.score);
console.log(ranked[0].name);   // 'Lucky' (score 98)
```

Chapter 7 — The DOM: Selecting, Manipulating & Traversing

The Document Object Model (DOM) is the browser's live representation of your HTML page as a tree of objects. JavaScript can read and modify any part of the DOM — changing text, styles, attributes, structure — and the page updates instantly without a reload.

7.1 Selecting Elements

```
// querySelector — returns the FIRST matching element
const title   = document.querySelector('h1');
const navLink = document.querySelector('nav a.active');
const hero    = document.querySelector('#hero');
const card    = document.querySelector('.card');

// querySelectorAll — returns ALL matching elements as a NodeList
const allLinks   = document.querySelectorAll('a');
const allButtons = document.querySelectorAll('.btn');

// Loop over NodeList
allButtons.forEach(btn => console.log(btn.textContent));

// Older methods (still common in legacy code)
document.getElementById('hero');        // by id
document.getElementsByClassName('card');// by class (live HTMLCollection)
document.getElementsByTagName('p');     // by tag (live HTMLCollection)
```

7.2 Reading & Changing Content

```
const title = document.querySelector('h1');

// Read content
console.log(title.textContent);   // Raw text (no HTML tags)
console.log(title.innerHTML);     // HTML content including tags

// Change text content (safe — no HTML injection risk)
title.textContent = 'Welcome to Get Techy With Lucky!';

// Change HTML content (allows injecting HTML tags)
const card = document.querySelector('.intro');
card.innerHTML = '<strong>JavaScript</strong> is the language of the web.';

// Read and set attributes
const img = document.querySelector('img');
console.log(img.getAttribute('alt'));
```

```
img.setAttribute('alt', 'Lucky teaching JavaScript at a workshop');

// Read and set input value
const emailInput = document.querySelector('#email');
console.log(emailInput.value);   // What the user typed
emailInput.value = '';           // Clear the field
```

7.3 Changing Styles & Classes

```
const box = document.querySelector('.card');

// Inline styles (use sparingly)
box.style.backgroundColor = '#1b3a6b';
box.style.color           = 'white';
box.style.padding         = '24px';

// classList — the RIGHT way to manage styles
box.classList.add('active');        // Add a class
box.classList.remove('hidden');     // Remove a class
box.classList.toggle('open');       // Add if absent, remove if present
box.classList.contains('active');   // Returns true/false
box.classList.replace('old','new'); // Replace one class with another
```

7.4 Creating & Removing Elements

```
// Create a new element
const newCard = document.createElement('article');
newCard.classList.add('card');
newCard.innerHTML = `
  <h3>New Course</h3>
  <p>Advanced JavaScript — starting next week.</p>
  <a href='/course' class='btn'>Enroll</a>
`;

// Add to the DOM
const grid = document.querySelector('.courses-grid');
grid.appendChild(newCard);          // Add at the END of grid
grid.prepend(newCard);              // Add at the BEGINNING of grid

// Insert relative to another element
const existingCard = document.querySelector('.card');
existingCard.insertAdjacentElement('afterend', newCard);

// Remove an element
newCard.remove();

// Remove a child
grid.removeChild(existingCard);
```

Chapter 8 — Events: Listeners, Types, Bubbling & Delegation

Events are actions that happen in the browser — a user clicking a button, typing in a field, scrolling the page, or the page finishing its load. JavaScript uses event listeners to detect these actions and run code in response.

8.1 addEventListener — The Right Way

```
const btn = document.querySelector('#submitBtn');

// addEventListener(eventType, handlerFunction)
btn.addEventListener('click', function() {
  console.log('Button was clicked!');
});

// Arrow function version (preferred)
btn.addEventListener('click', () => {
  btn.textContent = 'Submitted!';
  btn.disabled = true;
});

// Named function (reusable + removable)
function handleClick() {
  console.log('Handled!');
}
btn.addEventListener('click', handleClick);
btn.removeEventListener('click', handleClick);  // Remove later if needed
```

8.2 The Event Object

```
document.querySelector('#name').addEventListener('input', (event) => {
  console.log(event.type);          // 'input'
  console.log(event.target);        // The input element itself
  console.log(event.target.value);  // Current value as user types
});

document.addEventListener('keydown', (e) => {
  console.log(e.key);       // 'a', 'Enter', 'Escape', 'ArrowUp'...
  console.log(e.ctrlKey);   // true if Ctrl was held down
  if (e.key === 'Escape') closeModal();
});

document.addEventListener('mousemove', (e) => {
  console.log(`X: ${e.clientX}, Y: ${e.clientY}`);
```

```
});
```

8.3 Common Event Types

Event	When It Fires	Common Use
click	User clicks an element	Buttons, links, cards
dblclick	User double-clicks	Text select, open detail
mouseover	Mouse enters element area	Hover effects
mouseout	Mouse leaves element area	Remove hover effects
keydown	Key pressed down	Keyboard shortcuts
keyup	Key released	Live search, input tracking
input	Value changes in form field	Live validation, counters
change	Value confirmed (blur or select)	Select dropdowns, checkboxes
submit	Form submit button clicked	Form handling & validation
focus	Element receives focus	Highlight input field
blur	Element loses focus	Validate on leaving a field
scroll	Page or element is scrolled	Sticky header, lazy load
resize	Browser window resized	Responsive adjustments
DOMContentLoaded	HTML fully parsed (before images)	Run init code safely
load	Page fully loaded (images & all)	Show loader then content

8.4 Event Bubbling & stopPropagation

When an event fires on an element, it bubbles up through all its parent elements — firing on each one. This is called event bubbling.

```
<div id="outer">
  <div id="inner">
    <button id="btn">Click Me</button>
  </div>
</div>

// When btn is clicked, the click fires on: btn → inner → outer → document
document.querySelector('#btn').addEventListener('click', (e) => {
  console.log('Button clicked');
  e.stopPropagation();   // Stop bubbling — inner and outer won't fire
});

document.querySelector('#outer').addEventListener('click', () => {
  console.log('Outer clicked');   // Only fires if stopPropagation not called
});
```

8.5 Event Delegation — One Listener for Many Elements

```
// Instead of adding a listener to each card...
// Add ONE listener to the parent and check event.target

const courseGrid = document.querySelector('.courses-grid');

courseGrid.addEventListener('click', (e) => {
  // Check if a delete button was clicked
  if (e.target.matches('.btn-delete')) {
    const card = e.target.closest('.card');
    card.remove();
    console.log('Course card removed.');
  }

  // Check if an enroll button was clicked
  if (e.target.matches('.btn-enroll')) {
    const courseName = e.target.dataset.course;
    console.log(`Enrolling in: ${courseName}`);
  }
});

// This works even for cards added DYNAMICALLY after page load!
```

Chapter 9 — Forms & Validation with JavaScript

Forms are how users send data to websites. JavaScript makes forms interactive — validating inputs in real time, showing friendly error messages without page reloads, and controlling exactly what gets submitted.

9.1 Preventing Default Form Submission

```
const form = document.querySelector('#registerForm');

form.addEventListener('submit', (e) => {
  e.preventDefault();   // Stop the page from reloading

  // Now handle the form with JS
  const name  = document.querySelector('#name').value.trim();
  const email = document.querySelector('#email').value.trim();

  if (!name || !email) {
    showError('All fields are required.');
    return;
  }

  showSuccess(`Welcome, ${name}! Check ${email} for confirmation.`);
});
```

9.2 Complete Form Validation Example

```
form.addEventListener('submit', (e) => {
  e.preventDefault();
  clearErrors();
  let valid = true;

  const name     = document.querySelector('#name').value.trim();
  const email    = document.querySelector('#email').value.trim();
  const password = document.querySelector('#password').value;
  const confirm  = document.querySelector('#confirm').value;

  // Empty field check
  if (!name) { showFieldError('nameErr', 'Name is required.'); valid = false; }

  // Email format check
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailPattern.test(email)) {
    showFieldError('emailErr', 'Enter a valid email address.');
```

```
    valid = false;
  }

  // Password length
  if (password.length < 8) {
    showFieldError('pwdErr', 'Password must be at least 8 characters.');
    valid = false;
  }

  // Confirm password match
  if (password !== confirm) {
    showFieldError('confirmErr', 'Passwords do not match.');
    valid = false;
  }

  if (valid) {
    document.querySelector('#successMsg').textContent = `Account created for
${name}!`;
    form.reset();
  }
});

function showFieldError(id, message) {
  const el = document.getElementById(id);
  el.textContent = message;
  el.style.color = 'red';
}
function clearErrors() {
  document.querySelectorAll('.error').forEach(el => el.textContent = '');
}
```

9.3 Real-Time Validation as User Types

```
const passwordInput = document.querySelector('#password');
const strengthBar   = document.querySelector('#strengthBar');
const strengthLabel = document.querySelector('#strengthLabel');

passwordInput.addEventListener('input', () => {
  const pwd = passwordInput.value;
  let score = 0;

  if (pwd.length >= 8)             score++;
  if (/[A-Z]/.test(pwd))           score++;   // Has uppercase
  if (/[0-9]/.test(pwd))           score++;   // Has number
  if (/[^A-Za-z0-9]/.test(pwd))    score++;   // Has special char

  const levels = ['', 'Weak', 'Fair', 'Good', 'Strong'];
  const colors = ['', '#ef4444', '#f59e0b', '#3b82f6', '#22c55e'];

  strengthBar.style.width  = `${score * 25}%`;
```

```
  strengthBar.style.background = colors[score];
  strengthLabel.textContent    = levels[score];
});
```

Chapter 10 — ES6+ Modern JavaScript Features

ES6 (released in 2015) and subsequent versions introduced powerful new syntax that makes JavaScript cleaner, shorter, and more expressive. These features are used in every modern codebase — you must know them well.

10.1 let & const (Already Covered — Always Use These)

10.2 Destructuring Assignment

```
// Array destructuring
const [first, second, , fourth] = ['HTML', 'CSS', 'JS', 'PHP'];
console.log(first);   // 'HTML'
console.log(fourth);  // 'PHP'

// Swap variables using destructuring
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b);   // 2 1

// Object destructuring (already covered in Chapter 6)
const { name, age } = { name: 'Lucky', age: 30, city: 'Nyeri' };
```

10.3 Optional Chaining (?.)

```
const user = {
  name: 'Lucky',
  address: {
    city: 'Nyeri',
    // postcode not set
  }
};

// Without optional chaining — CRASHES if address is undefined
// console.log(user.address.postcode.code);  // TypeError!

// With optional chaining — returns undefined safely
console.log(user?.address?.postcode?.code);   // undefined (no crash)
console.log(user?.phone?.number);             // undefined

// Works with methods too
console.log(user.getName?.());  // undefined if method doesn't exist
```

10.4 Nullish Coalescing (??)

```
// ?? returns the RIGHT side only if LEFT is null or undefined
// (Unlike ||, which also triggers for 0, '', false)

const name = null ?? 'Guest';          // 'Guest'
const score = undefined ?? 0;          // 0
const count = 0 ?? 'No items';         // 0  (0 is NOT null/undefined)
const label = '' ?? 'Default';         // '' (empty string is NOT null/undefined)

// Practical use
const user = { name: 'Lucky', credits: 0 };
console.log(user.credits ?? 'No credits set');   // 0
console.log(user.credits || 'No credits set');   // 'No credits set' (wrong!)
```

10.5 Short-Circuit Evaluation

```
// && returns the FIRST falsy value, or the LAST value if all truthy
const isLoggedIn = true;
isLoggedIn && console.log('Show dashboard');   // Runs if isLoggedIn is true

// || returns the FIRST truthy value
const displayName = user.nickname || user.name || 'Anonymous';

// Conditional rendering pattern (common in React)
const errorMsg = hasError && `<p class='error'>${errorText}</p>`;
```

10.6 Map, Set, WeakMap, WeakSet

```
// Map — like an object but keys can be ANY type
const map = new Map();
map.set('name', 'Lucky');
map.set(1, 'one');
map.set(true, 'yes');
console.log(map.get('name'));   // 'Lucky'
console.log(map.size);          // 3

// Set — collection of UNIQUE values (no duplicates)
const set = new Set([1, 2, 3, 2, 1, 3, 4]);
console.log(set);           // Set {1, 2, 3, 4}
console.log(set.size);      // 4

// Practical: Remove duplicates from an array
const withDupes = [1, 2, 2, 3, 3, 4];
const unique = [...new Set(withDupes)];
console.log(unique);   // [1, 2, 3, 4]
```

Chapter 11 — Asynchronous JavaScript

JavaScript is single-threaded — it can only do one thing at a time. But web apps need to do many things at once: load data, handle clicks, run timers. Asynchronous JavaScript solves this by allowing time-consuming tasks to run in the background while the rest of the code continues.

11.1 The Problem: Synchronous vs Asynchronous

```
// SYNCHRONOUS — blocks until complete
console.log('1. Start');
// Imagine this takes 5 seconds to fetch data from a server...
// Everything else WAITS.
console.log('2. Done');

// ASYNCHRONOUS — does not block
console.log('1. Start');
setTimeout(() => {
  console.log('3. This ran after 2 seconds');
}, 2000);
console.log('2. This runs immediately — does not wait!');
// Output: 1. Start  →  2. This runs immediately  →  3. This ran after 2 seconds
```

11.2 Callbacks

```
// A callback is a function passed as an argument and called later
function fetchUser(userId, onSuccess, onError) {
  setTimeout(() => {
    if (userId === 1) {
      onSuccess({ id: 1, name: 'Lucky' });
    } else {
      onError('User not found');
    }
  }, 1000);
}

fetchUser(1,
  (user) => console.log(`Found: ${user.name}`),    // Success callback
  (err)  => console.log(`Error: ${err}`)           // Error callback
);
```

11.3 Promises

```
// A Promise represents a value that will be available in the future
// States: pending → fulfilled (resolved) or rejected

const fetchData = new Promise((resolve, reject) => {
```

```
  setTimeout(() => {
    const success = true;
    if (success) {
      resolve({ name: 'Lucky', course: 'JavaScript' });
    } else {
      reject(new Error('Network request failed'));
    }
  }, 1500);
});

fetchData
  .then(data  => console.log(`Success: ${data.name}`))   // When resolved
  .catch(err  => console.log(`Error: ${err.message}`))   // When rejected
  .finally(() => console.log('Request complete.'));       // Always runs
```

11.4 async / await — The Cleanest Syntax

async/await is built on top of Promises but reads like synchronous code. It is the most modern and readable way to handle asynchronous operations.

```
// Mark the function as async
async function loadUserData(userId) {
  try {
    // await pauses HERE until the Promise resolves
    const response = await fetch(`https://api.example.com/users/${userId}`);

    if (!response.ok) {
      throw new Error(`HTTP Error: ${response.status}`);
    }

    const user = await response.json();
    console.log(`Loaded: ${user.name}`);
    return user;

  } catch (error) {
    console.error(`Failed to load user: ${error.message}`);
  }
}

// Call it
loadUserData(1);
```

11.5 The Fetch API — Calling Real APIs

```
// GET request — fetch data from an API
async function getPosts() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts');
  const posts = await response.json();
```

```
  posts.slice(0, 3).forEach(post => {
    console.log(`${post.id}. ${post.title}`);
  });
}

// POST request — send data to an API
async function createPost(title, body) {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ title, body, userId: 1 })
  });
  const newPost = await response.json();
  console.log(`Created post with ID: ${newPost.id}`);
}
```

Chapter 12 — Error Handling & Debugging

12.1 try / catch / finally

```
function parseUserAge(input) {
  try {
    const age = parseInt(input);
    if (isNaN(age)) throw new Error('Input is not a valid number.');
    if (age < 0 || age > 120) throw new RangeError('Age must be 0-120.');
    return age;
  } catch (error) {
    if (error instanceof RangeError) {
      console.error('Range Error:', error.message);
    } else {
      console.error('General Error:', error.message);
    }
    return null;
  } finally {
    console.log('parseUserAge() completed.');  // Always runs
  }
}

parseUserAge('25');     // Returns 25
parseUserAge('abc');    // Logs: General Error: Input is not a valid number.
parseUserAge('200');    // Logs: Range Error: Age must be 0-120.
```

12.2 Common JavaScript Errors

Error Type	Description & Example
ReferenceError	Using a variable that has not been declared: console.log(undeclaredVar)
TypeError	Calling something that is not a function, or accessing a property on null/undefined.
SyntaxError	Code that cannot be parsed — missing brackets, semicolons, mismatched quotes.
RangeError	Value outside of allowed range — e.g. new Array(-5)
URIError	Incorrect use of URI encoding functions.
EvalError	Issues with the eval() function (rarely encountered).

12.3 Debugging with Chrome DevTools

6. Press F12 (or Ctrl+Shift+I) to open Developer Tools.
7. Click the Console tab to see errors, logs, and run JS commands live.

8. Click the Sources tab to view your JS files and add breakpoints.
9. Click on a line number in Sources to add a breakpoint — execution pauses there.
10. Use 'Step Over' (F10) and 'Step Into' (F11) to walk through code line by line.
11. Hover over variables while paused to inspect their current values.
12. In Console, type any variable name to inspect its value at that moment.

DEBUGGING TOOLKIT

```
console.log(value)           — print a value to check it
console.error('message')     — print a red error message
console.warn('message')      — print a yellow warning
console.table(arrayOfObjects) — display data in a formatted table
console.group('label')       — group related logs together
debugger;                    — pauses execution at this line (DevTools must be open)
typeof value                 — check the data type of any value
JSON.stringify(obj, null, 2)  — pretty-print an object for inspection
```

Chapter 13 — JavaScript in the Browser: Storage, Timers & APIs

13.1 Timers

```
// setTimeout — run code ONCE after a delay
const timer = setTimeout(() => {
  console.log('This runs after 3 seconds.');
  document.querySelector('.toast').classList.remove('show');
}, 3000);

// Cancel a timeout before it fires
clearTimeout(timer);

// setInterval — run code REPEATEDLY every N milliseconds
let count = 0;
const interval = setInterval(() => {
  count++;
  document.querySelector('#counter').textContent = count;
  if (count >= 10) clearInterval(interval);  // Stop after 10
}, 1000);
```

13.2 Web Storage: localStorage & sessionStorage

```
// localStorage — persists FOREVER (even after browser closes)
localStorage.setItem('theme', 'dark');
localStorage.setItem('username', 'Lucky');

const theme    = localStorage.getItem('theme');      // 'dark'
const username = localStorage.getItem('username');   // 'Lucky'

localStorage.removeItem('theme');   // Remove one item
localStorage.clear();               // Remove ALL items

// Store objects — must JSON.stringify() first
const user = { name: 'Lucky', role: 'admin', visits: 42 };
localStorage.setItem('user', JSON.stringify(user));

// Retrieve and parse
const storedUser = JSON.parse(localStorage.getItem('user'));
console.log(storedUser.name);   // 'Lucky'

// sessionStorage — same API, but cleared when browser tab closes
sessionStorage.setItem('tempToken', 'abc123');
```

13.3 URL & Query Strings

```
// Read the current page URL
console.log(window.location.href);      // Full URL
console.log(window.location.hostname);  // 'example.com'
console.log(window.location.pathname);  // '/courses/javascript'
console.log(window.location.search);    // '?id=42&tab=overview'

// Parse query parameters
const params = new URLSearchParams(window.location.search);
console.log(params.get('id'));    // '42'
console.log(params.get('tab'));   // 'overview'

// Navigate programmatically
window.location.href = '/dashboard';
history.back();     // Go back one page
history.forward();  // Go forward one page
```

13.4 JSON — JavaScript Object Notation

```
// JSON is a text format for storing and transporting data
// JavaScript objects  ⟷  JSON strings

const person = { name: 'Lucky', age: 30, skills: ['HTML', 'CSS'] };

// Object to JSON string
const jsonString = JSON.stringify(person);
console.log(jsonString);
// '{"name":"Lucky","age":30,"skills":["HTML","CSS"]}'

// Pretty-print with indentation
console.log(JSON.stringify(person, null, 2));

// JSON string back to object
const parsed = JSON.parse(jsonString);
console.log(parsed.name);   // 'Lucky'
console.log(parsed.skills); // ['HTML', 'CSS']
```

Chapter 14 — Object-Oriented JavaScript: Classes & Prototypes

Object-Oriented Programming (OOP) is a way of organising code around 'objects' that have properties (data) and methods (behaviour). JavaScript supports OOP through classes (ES6) and prototypes (older style).

14.1 Classes — ES6 Style

```
class Student {
  // Constructor runs when you create a new instance
  constructor(name, email, course) {
    this.name   = name;
    this.email  = email;
    this.course = course;
    this.marks  = [];
  }

  // Method
  addMark(mark) {
    this.marks.push(mark);
  }

  // Getter
  get average() {
    if (this.marks.length === 0) return 0;
    const sum = this.marks.reduce((a, b) => a + b, 0);
    return (sum / this.marks.length).toFixed(1);
  }

  // Method
  describe() {
    return `${this.name} | Course: ${this.course} | Average: ${this.average}%`;
  }
}

// Create instances
const s1 = new Student('Lucky', 'lucky@example.com', 'JavaScript');
s1.addMark(88);
s1.addMark(95);
s1.addMark(79);
console.log(s1.describe());
// Lucky | Course: JavaScript | Average: 87.3%
```

14.2 Inheritance with extends

```
class Person {
  constructor(name, email) {
    this.name  = name;
    this.email = email;
  }
  greet() { return `Hello, I am ${this.name}.`; }
}

// Student EXTENDS Person — inherits all properties and methods
class Student extends Person {
  constructor(name, email, course) {
    super(name, email);   // Call parent constructor FIRST
    this.course = course;
  }
  // Override the greet method
  greet() {
    return `${super.greet()} I study ${this.course}.`;
  }
}

class Instructor extends Person {
  constructor(name, email, subject) {
    super(name, email);
    this.subject = subject;
  }
  teach() { return `${this.name} is teaching ${this.subject}.`; }
}

const lucky = new Instructor('Lucky Nakola', 'lucky@example.com', 'JavaScript');
console.log(lucky.greet());    // Hello, I am Lucky Nakola.
console.log(lucky.teach());   // Lucky Nakola is teaching JavaScript.
```

Chapter 15 — Modules, Best Practices & Next Steps

15.1 JavaScript Modules (ES6)

Modules allow you to split your JavaScript into multiple files and share code between them using export and import.

```
// === math.js — EXPORT functions ===
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
export const PI = 3.14159;

// Default export (one per file)
export default function multiply(a, b) { return a * b; }



// === app.js — IMPORT from math.js ===
import multiply, { add, subtract, PI } from './math.js';

console.log(add(5, 3));        // 8
console.log(subtract(10, 4));  // 6
console.log(multiply(6, 7));   // 42
console.log(PI);               // 3.14159



// In HTML — use type='module' to enable ES modules
// <script type='module' src='app.js'></script>
```

15.2 JavaScript Best Practices

Best Practice	Why It Matters
Use const by default	Reach for let only when you need to reassign. Never use var.
Always use ===	Strict equality prevents type-coercion bugs.
Meaningful variable names	let userAge not let a. let emailAddress not let ea.
One responsibility per function	Each function should do ONE thing well.
Keep functions small	If a function is longer than 20 lines, consider breaking it up.
Handle errors	Always use try/catch around async code and risky operations.
Comment the WHY not the WHAT	// Clamp to max 5 retries — not // increment i

Best Practice	Why It Matters
Avoid global variables	Wrap code in functions or modules to prevent polluting the global scope.
Validate all inputs	Never trust data from the user — validate on both frontend and backend.
DRY — Don't Repeat Yourself	If you write the same code twice, put it in a function.
Use template literals	Much cleaner than string concatenation with +.
Use array methods over loops	map, filter, reduce are cleaner and more expressive than for loops.
Semicolons	Always add semicolons at the end of statements to avoid ASI edge cases.
Run code in strict mode	'use strict'; at the top catches silent errors.
Use a linter (ESLint)	Automatically flags bad practices and style inconsistencies.

15.3 Your JavaScript Learning Roadmap

Stage	Topics
NOW (this course)	Variables, data types, operators, control flow, functions, arrays, objects, DOM, events, forms
NEXT	Async JS, Fetch API, ES6+, classes, modules, error handling, localStorage
THEN — Frameworks	React (most popular), Vue.js (beginner-friendly), Angular (enterprise)
THEN — Backend	Node.js + Express for server-side JS. Connect JS to MySQL or MongoDB.
THEN — Tools	Git & GitHub, npm, Webpack/Vite, TypeScript
ADVANCED	Testing (Jest), Performance, Security, PWAs, WebSockets, GraphQL

Chapter 16 — Comprehensive Quiz & Coding Challenges

Test your full understanding of JavaScript. Attempt every section independently before reviewing the answers.

Section A — Multiple Choice (25 questions — 1 mark each)

1. JavaScript was originally created in how many days?
   a) 30 days b) 10 days c) 1 year d) 6 months

2. Which keyword declares a variable that CANNOT be reassigned?
   a) let b) var c) const d) fixed

3. What does typeof [] return in JavaScript?
   a) 'array' b) 'list' c) 'object' d) 'undefined'

4. Which comparison operator checks BOTH value AND type?
   a) == b) != c) === d) =

5. What is the output of: console.log(10 % 3);
   a) 3 b) 1 c) 3.33 d) 0

6. Which of these is a correct arrow function?
   a) function => (x) { return x * 2; } b) (x) => x * 2
   c) arrow (x) { return x * 2; } d) x -> x * 2

7. What does the splice() method do to an array?
   a) Returns a new array without changing the original
   b) Splits the array at a delimiter
   c) Modifies the original array by removing or inserting elements
   d) Searches the array and returns the index

8. Which array method creates a NEW array by transforming every element?
   a) forEach b) filter c) find d) map

9. What does filter() return when NO elements pass the test?
   a) null b) undefined c) An empty array [] d) false

10. Which method removes the LAST element from an array and returns it?

    a) shift() b) splice() c) pop() d) remove()

11. How do you safely access a deeply nested property that might be undefined?

    a) user.address.city b) user?.address?.city
    c) user['address']['city'] d) get(user, 'address.city')

12. What does e.preventDefault() do in a form submit handler?

    a) Stops the page from reloading when the form is submitted
    b) Prevents the form from validating
    c) Clears all form fields
    d) Removes the event listener

13. What is the DOM?

    a) A database for storing JavaScript objects
    b) The browser's live tree representation of the HTML page
    c) A JavaScript framework for building UIs
    d) A method for loading CSS files

14. Which method selects ALL elements matching a CSS selector?

    a) document.getElementById() b) document.querySelector()
    c) document.querySelectorAll() d) document.selectAll()

15. What does classList.toggle('active') do?

    a) Always adds the class 'active'
    b) Always removes the class 'active'
    c) Adds the class if absent, removes it if present
    d) Checks if the element has the class and returns a boolean

16. Which event fires continuously as the user types in an input field?

    a) change b) keypress c) input d) type

17. What is event bubbling?

    a) When an event fires multiple times on the same element
    b) When an event on a child element propagates up through its parent elements
    c) When two events fire simultaneously
    d) When an event is cancelled before reaching its target

18. Which method is used to send a GET request to an API in modern JavaScript?

    a) request() b) http.get() c) ajax() d) fetch()

19. What are the three states of a JavaScript Promise?

a) start, running, done
b) pending, fulfilled, rejected
c) loading, success, error
d) created, executed, destroyed

20. What does the await keyword do inside an async function?

a) Skips the next line of code
b) Pauses execution until the Promise resolves or rejects
c) Creates a new Promise automatically
d) Converts the function to synchronous

21. Which storage option persists data AFTER the browser tab is closed?

a) sessionStorage b) cookies only c) localStorage d) memoryStorage

22. What is the correct way to convert a JavaScript object to a JSON string?

a) JSON.parse(obj) b) JSON.convert(obj)
c) JSON.stringify(obj) d) obj.toJSON()

23. Which keyword is used to call the parent class constructor inside a subclass?

a) this() b) parent() c) extends() d) super()

24. What does the spread operator (...) do when used with an array?

a) Creates a deep copy of the array
b) Expands the array into individual elements
c) Removes duplicate values from the array
d) Sorts the array in ascending order

25. Which of the following correctly imports a named export from './utils.js'?

a) import utils from './utils.js'
b) import { formatDate } from './utils.js'
c) require('./utils.js').formatDate
d) include { formatDate } from './utils.js'

Section B — Short Answer (10 questions — 3 marks each)

1. Explain the difference between let, const, and var. When would you use each one? Why should var be avoided in modern JavaScript?

2. What is the difference between == and ===? Give an example where == produces an unexpected result but === does not.

3. Explain what a higher-order function is. Write a simple example showing a function that takes another function as an argument and calls it.

4. What is the difference between map() and forEach()? When would you choose map() over forEach()? Give an example of each.

5. Explain event delegation. Why is it better to use event delegation on a parent element than attaching event listeners to every child element individually?

6. What is a Promise? Describe its three states. Write a simple Promise that resolves with a user's name after 1 second.

7. Explain the difference between localStorage and sessionStorage. Write code to save an object to localStorage and then read it back.

8. What is a JavaScript class? Write a class called 'BankAccount' with properties (owner, balance), a method to deposit money, a method to withdraw (check if funds are available), and a method to return the current balance.

9. What is the optional chaining operator (?.) and why is it useful? Give an example of code that would crash without it and show how ?. prevents the crash.

10. Explain what async/await does. Why is it preferred over raw .then()/.catch() Promise chaining for reading code?

Section C — Coding Challenges (Write complete, working JavaScript)

Challenge 1 — DOM & Events:

Create an HTML page with a list of items (at least 5). Using JavaScript: (a) Add a 'Delete' button to each list item dynamically using a loop. (b) Use event delegation on the parent <ul> to listen for delete button clicks. (c) When a delete button is clicked, remove that list item with a smooth fade-out animation using CSS classes and setTimeout. (d) Show a count of remaining items at the top of the list — update it each time an item is removed.

Challenge 2 — Arrays & Objects:

You have an array of student objects: [{name:'Alice', marks:[78,85,91]}, {name:'Bob', marks:[55,60,72]}, {name:'Carol', marks:[92,88,95]}, {name:'David', marks:[40,45,38]}]. Write JavaScript to: (a) Calculate the average mark for each student. (b) Assign a grade (A=80+, B=65+,

C=50+, F=below 50) based on the average. (c) Filter out only the students who passed (grade not F). (d) Sort the passing students by average mark (highest first). (e) Print a formatted report using template literals for each passing student.

Challenge 3 — Async & Fetch:

Write an async JavaScript function called 'fetchAndDisplay()' that: (a) Fetches posts from 'https://jsonplaceholder.typicode.com/posts?_limit=5' using the Fetch API. (b) Handles any network errors using try/catch. (c) Creates an HTML card for each post in the DOM (title, body, post ID). (d) Shows a loading spinner while the data loads (add/remove a CSS class). (e) Displays a user-friendly error message in the DOM if the fetch fails — no alert() allowed.

Section A — Answer Key

Multiple Choice Answers

1. b) 10 days 2. c) const 3. c) 'object'
4. c) === 5. b) 1 6. b) (x) => x * 2
7. c) Modifies original (remove/insert) 8. d) map
9. c) An empty array [] 10. c) pop()
11. b) user?.address?.city 12. a) Stops page from reloading
13. b) Browser's live tree of the HTML page 14. c) querySelectorAll()
15. c) Adds if absent, removes if present 16. c) input
17. b) Event propagates up through parents 18. d) fetch()
19. b) pending, fulfilled, rejected 20. b) Pauses until Promise resolves
21. c) localStorage 22. c) JSON.stringify(obj)
23. d) super() 24. b) Expands into individual elements
25. b) import { formatDate } from './utils.js'

The best way to learn JavaScript is to write JavaScript. Build something today.

Get Techy With Lucky | Tech Pulse Insider | Instructor: Lucky Nakola